

Test Case Prioritization for Black Box Testing^{*}

Bo Qu Changhai Nie Baowen Xu Xiaofang Zhang
School of Computer Science and Engineering
Southeast University, Nanjing, China
{boqu changhainie, bwxu, xfzhang}@seu.edu.cn

Abstract

Test case prioritization is an effective and practical technique that helps to increase the rate of regression fault detection when software evolves. Numerous techniques have been reported in the literature on prioritizing test cases for regression testing. However, existing prioritization techniques implicitly assume that source or binary code is available when regression testing is performed, and therefore cannot be implemented when there is no program source or binary code to be analyzed. In this paper, we presented a new technique for black box regression testing, and we performed an experiment to measure our technique. Our results show that the new technique is helpful to improve the effectiveness of fault detection when performing regression test in black box environment.

Key words: Regression testing, black box testing, prioritization, test history

1. Introduction

Regression testing is a necessary but expensive process in software lifecycle. One of regression testing approach, test case prioritization, aims at sorting and executing test case in the order of their potential abilities to achieve certain testing objective. This approach reuses test suite and assigns each test case a selection probability according to its historical performance, and then selects ordered test cases to run until testing resources exhaust. Empirical studies indicate that test case prioritization technique could increase the effectiveness of test cases at meeting some performance goals especially in resource-constrained testing environment[3].

Numerous techniques have been reported in the literature on prioritizing test cases for regression testing[1,2,5,6,9-15]. Amongst these techniques, some studies incorporate source code information, and others investigate prioritization techniques by binary code comparison[13]. However, none of existing techniques could be of use when source code or binary code is unavailable. To apply test case prioritization in such situation, practitioners need a new technique for black box testing. To date, no such technique has been provided.

In this paper, we present a general process of test case prioritization for regression testing in black box environment. We implement algorithms based on test history and run-time information. The main idea of these algorithms is to group all reused test cases by the fault types they have revealed, and then dynamically adjust their priorities according to test results of related test cases. In order to measure their abilities in detecting regression faults, we also perform an experiment. The results show that our algorithms can help improving the test suite's rate of fault detection.

The rest of this article is organized as follows: Section 2 reviews the regression testing and test case prioritization problem, and summarizes related work. Section 3 presents details of our work, including a general test case prioritization process for black box testing and corresponding algorithms. Section 4 presents the design and analysis of our experiment. Section 5 concludes and discusses future work.

2. Background

2.1. Test case prioritization

Test case prioritization schedules test cases so that those with the highest priority, according to some criterion, are executed earlier in the regression testing process than lower priority test cases[4]. Kim et al indicate that a typical test case prioritization process works as follows[6]: (1) apply a regression test

^{*} This work was supported in part by the National Natural Science Foundation of China (60425206, 60633010, 60403016, 60503033), Natural Science Foundation of Jiangsu Province (BK2005060), High Technology Research Project of Jiangsu Province (BG2005032), Excellent Talent Foundation on Teaching and Research of Southeast University.

selection technique on test suite T , yielding T' , (2) assign a selection probability to each test case in T' , (3) draw a test case from T' using the probabilities assigned in step 2, and run it, and (4) repeat step 3 until testing time is exhausted. Rothermel et al have formally defined step 2, the test case prioritization problem, as follows[11]:

Definition 1: The Test Case Prioritization Problem

Given: T , a test suite; PT , the set of permutations of T ; f , a function from PT to the real numbers.

Problem: Find $T' \in PT$ such that

$$(\forall T'')(T'' \in PT)(T'' \neq T') [f(T') \geq f(T'')].$$

Here, PT represents the set of all possible orderings of T , and f is a function which applied to any such ordering.

Although there are many possible goals for prioritization, we restrict our attention to techniques that would improve test suite's efficiency of detecting regression faults. To formally illustrate how rapidly a prioritized test suite detects faults, Rothermel et al introduced a metric called *APFD* to measure the weighted average of the percentage of faults detected during the execution of the test suite[6]. Let T be a test suite which contains n test cases, and let F be a set of m faults revealed by T . Let TF_i be the first test case in ordering T' of T which reveals fault i . According to [4], the *APFD* for test suite T' could be given by the equation:

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n}$$

APFD value ranges from 0 to 100. An ordered test suite with higher *APFD* value has faster (better) fault detection rates than those with lower *APFD* values.

2.2. Related work

To improve the faults detection rate of test suite, researchers have considered the problem of test case prioritization. Many techniques[1,2,5,6,9-15] and empirical studies[3,4] have been reported in the literature on prioritizing test cases for regression testing in recent years.

Wong et al proposed a hybrid technique which combines modification, minimization and prioritization based selection using source code changes and test history[15]. They also suggested that prioritization approach could be especially useful when testers can only afford to rerun a few regression tests. Elbaum et al indicated that prioritization techniques could significantly improve rate of fault detection[1]. Rothermel et al formally defined the test case prioritization problem and presented metrics for measuring the rate of fault detection of test suites[11]. They also presented several test case prioritization

techniques based on source code and test history. Elbaum et al proposed an improved test case prioritization technique by incorporating varying test costs and fault severities[2]. Jones et al presented a technique which prioritizes test cases for use with the modified condition/decision coverage (MC/DC) criteria[5]. Srivastava et al described a test case prioritization technique based on basic block coverage for large systems[13]. Kim et al modeled regression testing as an ordered sequence of testing sessions and presented a history based prioritization technique which utilizes the information from previous testing[6]. Elbaum et al presented empirical study results of selecting cost-effective test case prioritization techniques in different test scenarios[4]. Srikanth proposed a test case prioritization technique based on requirements[12]. Walcott et al presented an approach to prioritization incorporates a testing time budget[14].

Among the papers described above, most techniques rely on code coverage. However, when testing is performed in a black box environment, practitioners cannot acquire code coverage and other source code related information. Therefore, it is hard to apply existing techniques when source code is not available. To apply test case prioritization to such situation, practitioners need a new technique for black box testing.

3. Prioritization for black box testing

When testing in black box environment, source code related information is not available any more. In such situation, practitioners can only get outputs of test cases and other run-time information, such as running time of test case. Since only limited information could be used to reorder test cases, we proposed a prioritization technique based on test results and run-time information.

3.1. General process

One general method of prioritization for black box testing is to initialize test suite using test history, and then adjust the orders of the rest test cases based on run-time information. To guide the adjusting strategy, a matrix R is used. We define this matrix to predict the fault detection relationship of test cases, so once a test case revealed regression faults, related test cases can be adjusted to higher priority to acquire better rate of fault detection. Let T be a test suite, let T' be a subset of T , and let R be a matrix which describes the fault detection relationship of test cases. We describe the general process of test case prioritization for black box testing as follows:

1. Select T' from T and prioritize T' using available test history.
2. Build test cases relation matrix R based on available information.
3. Draw a test case from T' , and run it.
4. Reorder rest test cases using run-time information and test case relation matrix R .
5. Repeat step 3 until testing resource is exhausted.

In performing these steps, two main problems should be addressed. One is the algorithm of building test case relation matrix R mentioned in step 2. Another is the adjusting algorithm of using R to reorder test cases described in step 4.

To implement the prioritization process for black box testing, we should firstly apply regression testing selection technique to original test suite and initialize the selected test suite. One method to achieve this goal is selecting all test cases that revealed regression faults, and then ordering test cases by their prior performance of detecting faults. In this paper, therefore, we will restrict our attention to the definition and usage of R in step 2 and 4 described above.

3.2. Test case prioritization

As described in 3.1, matrix R predicts the fault detection relationship of test cases, so once a test case revealed regression faults, related test cases can be adjusted to higher priority to acquire better rate of fault detection. An ideal R can accurately describe the relationship between each two test cases. However, it is difficult to build such an ideal matrix before regression testing takes place. Instead, we present a test history based method to construct R . Let T' be a test suite which contains n test cases $t_1, t_2, \dots, t_n$, let F_i be the fault type that t_i reveals, let R be a $n \times n$ Boolean matrix, and let R_{ij} be the element that describes the relation between t_i and t_j , Figure 1 presents the algorithm of constructing R .

```

algorithm      BuildRelationMatrix( $T', F$ ): $R$ 
input           $T'$ : the selected test suite
                   $F$ : the types of faults that revealed in last test
output          $R$ : the relation matrix
1. begin
2.   for  $i = 1$  to  $n$ 
3.     for  $j = 1$  to  $n$            //enumerate all test cases
4.       if  $F_i = F_j$  then       //if they have reveal the same type of faults
5.          $R_{ij} = \text{true}$          //set to true
6.       else
7.          $R_{ij} = \text{false}$        // otherwise, set to false
8.       end if
9.     end for
10.  end for
11.  return  $R$ 
12. end

```

Figure 1. Algorithm for building relation matrix

For simplicity, we assume that each test can only reveal one type of regression fault. According to R , test cases are grouped by the fault types they have revealed. Test cases in one group may have same fault detecting abilities due to their same test histories. In other words, when regression testing is performing, once a test case detected a fault in modified program, it may imply that the modified version contains same type of faults. Therefore, test cases in one group should be set to higher priorities to detecting these potential faults earlier.

Based upon the relation matrix, one regression testing strategy is to select one test case to run, and then escalate or de-escalate the priorities of its related test cases that have not been used. To describe the adjusting scope, we defined a sensitive constant which indicates the maximum adjusting range. Test case to be adjusted would be set to higher or lower order within this range. Let α be the sensitive constant, two adjusting algorithms are showed in Figure 2 and 3.

Algorithm **Escalate** deals with all test cases that have not been used in their ordinal order. When t_m is tested, test case $m+1$ will be tested with the highest priority, so it is not necessary to escalate it any more. If the test case is a related one, algorithm will raise its priority with the maximum range of α . The algorithm checks if there are other related test cases in range, then adjusts the priority. It ensures that the test case won't be adjusted to higher priority than other related test cases, so the orders of test cases which are in the same group are kept.

Algorithm **Deescalate** deals with all rest test cases in reverse order. Since test case n has the lowest priority, it is not necessary to deescalate it any more. If the test case is a related one, algorithm will lower its priority with the max range of α . The algorithm also ensures that the test case won't be adjusted to lower priority than other related test cases in order to keep the order of test cases which are in the same group.

algorithm Escalate(T', R, m): T''
input T' : the selected test suite
 R : the relation matrix
 m : the index of current test case
 T'' : the adjusted test suite

output

```

1. begin
2.   for  $i = m+2$  to  $n$ 
3.      $range = \text{Min}(i-m-1, \alpha)$ 
4.     if  $R_{mi} = \text{true}$  then
5.       for  $j = i-1$  to  $i-range$  step -1
6.         if  $R_{mj} = \text{true}$  then
7.            $j = j + 1$ 
8.           break
9.         end if
10.      end for
11.      for  $k = i-1$  to  $j$  step -1
12.        Swap  $t_{k+1}$  and  $t_k$ 
13.      end for
14.    end if
15.  end for
16.  return  $T''$ 
17. end

```

Figure 2. Algorithm for escalating priority

algorithm Deescalate(T', R, m): T''
input T' : the selected test suite
 R : the relation matrix
 m : the index of current test case
 T'' : the adjusted test suite

output

```

1. begin
2.   for  $i = n-1$  to  $m+1$  step -1
3.      $range = \text{Min}(n-i, \alpha)$ 
4.     if  $R_{mi} = \text{true}$  then
5.       for  $j = i+1$  to  $i+range$ 
6.         if  $R_{mj} = \text{true}$  then
7.            $j = j - 1$ 
8.           break
9.         end if
10.      end for
11.      for  $k = i+1$  to  $j$ 
12.        Swap  $t_{k+1}$  and  $t_k$ 
13.      end for
14.    end if
15.  end for
16.  return  $T''$ 
17. end

```

Figure 3. Algorithm for de-escalating priority

	t1	t2	t3	t4	t5	t6
t1	1	0	1	0	0	1
t2	0	1	0	1	0	0
t3	1	0	1	0	0	1
t4	0	1	0	1	0	0
t5	0	0	0	0	1	0
t6	1	0	1	0	0	1

A. Relation matrix R

Step	Order
1	t3, t2, t4, t5, t6
2	t3, t2, t4, t5, t6
3	t3, t2, t4, t5, t6
4	t3, t2, t6, t4, t5

B. Escalate algorithm

Step	Order
1	t2, t3, t4, t5, t6
2	t2, t3, t4, t5, t6
3	t2, t4, t5, t3, t6
4	t2, t4, t5, t3, t6

C. Deescalate algorithm

Figure 4. Example illustrating the two algorithms

Figure 4 presents an example to illustrate how the two algorithms work. The initial order of test cases is t2, t3, t4, t5, t6. Sensitive constant here is 2. Figure 4.A depicts the relation matrix R . For simplicity, we use 1 and 0 to represent true and false. Figure 4.B and 4.C shows the processes when escalate and deescalate algorithms are implemented based on t1. As observed in Figure 4.A, t1, t3 and t6 are in one group; t2 and t4 are in another group. So based on t1, test case t3 and t6 would be adjusted.

In Figure 4.B, since t2's priority is the highest, it is t3 that the first test case which the escalate algorithm deals with. Because t1 and t3 are in same group, t3 is adjusted to higher position. Test case t4 and t5 are not related test cases, so they won't be adjusted. And t6 is adjusted to higher position with the range of 2.

In Figure 4.C, the deescalate algorithm deals with all test cases in reverse order. Test case t6 is ignored, t5, t4

and t2 are not adjusted. Only t3's priority is deescalated.

In terms of different test results, there are two test strategies rely on run-time information and relation matrix. The first strategy focuses on test cases which reveal faults by calling Escalate when test case failed. The second one addresses test cases that don't reveal faults by calling Deescalate when test case passed. Figure 5 and 6 present two algorithms when corresponding strategies applied on the general process described in 3.1.

The algorithms BBR1 and BBR2 call function BuildRelationMatrix for one time, and call function Escalate or Deescalate for n times, so the time complexities of two algorithms are both $O(n^2)$. Although our algorithms involve the use of feedback, their time complexities are independent from the size of programs to be tested.

```

algorithm    BBR1( $T'$ ,  $F$ ):  $T''$ 
input         $T'$ : the selected test suite
                $F$ : the types of faults that revealed in last test
output        $T''$ : the test cases that failed in regression testing
1.  begin
2.     $R = \text{BuildRelationMatrix}(T', F)$  //build the relation matrix
3.     $T'' = \phi$  //initialize return value
4.    for  $i = 0$  to  $n$  //adjust rest test cases. Ignore  $t_n$ 
5.      if test resource is exhausted then
6.        return  $T''$  //return when test resource is exhausted
7.       $\text{result} = \text{Run}(t_i)$  //choose the test case with highest priority to run
8.      if  $\text{result} = \text{false}$  then //when test case failed
9.         $T'' = T'' + t_i$  //add to  $T''$ 
10.      $T' = \text{Escalate}(T', R, i)$  //adjust priorities of remain test cases
11.      end if
12.    end for
13.    return  $T''$ 
14. end

```

Figure 5. Algorithm concerning failed test cases

```

algorithm    BBR2( $T'$ ,  $F$ ):  $T''$ 
input         $T'$ : the selected test suite
                $F$ : the types of faults that revealed in last test
output        $T''$ : the test cases that failed in regression testing
1.  begin
2.     $R = \text{BuildRelationMatrix}(T', F)$  //build the relation matrix
3.     $T'' = \phi$  //initialize return value
4.    for  $i = 0$  to  $n$  //adjust rest test cases. Ignore  $t_n$ 
5.      if test resource is exhausted then
6.        return  $T''$  //return when test resource is exhausted
7.       $\text{result} = \text{Run}(t_i)$  //choose the test case with highest priority to run
8.      if  $\text{result} = \text{false}$  then //when test case failed
9.         $T'' = T'' + t_i$  //add to  $T''$ 
10.     else
11.        $T' = \text{Deescalate}(T', R, i)$  //adjust priorities of remain test cases
12.     end if
13.    end for
14.    return  $T''$ 
15. end

```

Figure 6. Algorithm concerning passed test cases

4. Experiments

To assess whether our algorithms help to improve the effectiveness when performing regression test in black box environment, we carried out an experiment to measure the rate of detecting regression faults.

4.1. Effectiveness measure

To estimate the performance of our algorithm, we require a measure with which we could assess and compare different regression testing strategies in black box. In previous literature, Rothmel et al have introduced a metric called *APFD* to measure the weighted average of the percentage of faults detected during the execution of the test suite. However, this

metric involves fault numbers that each test case reveals. Since is hard to acquire such information when testing in a black box environment, it is difficult to implement. In such situation, as discussed before, practitioners can acquire outputs of test cases and some run-time information. Based on these, we present a new metric called M_0 as a measure to illustrate how rapidly an ordered test suite detects faults in black box testing.

In order to illustrate this measure, we present an example test suite with its ability of detecting regression fault. Figure 7 shows the different M_0 values of different test order.

Figure 7.A shows the test suite and its test history. Figure 7.B and 7.C reflect the situations of different test orders. M_0 here is represented by the percentage of shadowed area. When test order is *B-E-C-A-D*, test suite detects faults in the 17th second, 19th second and

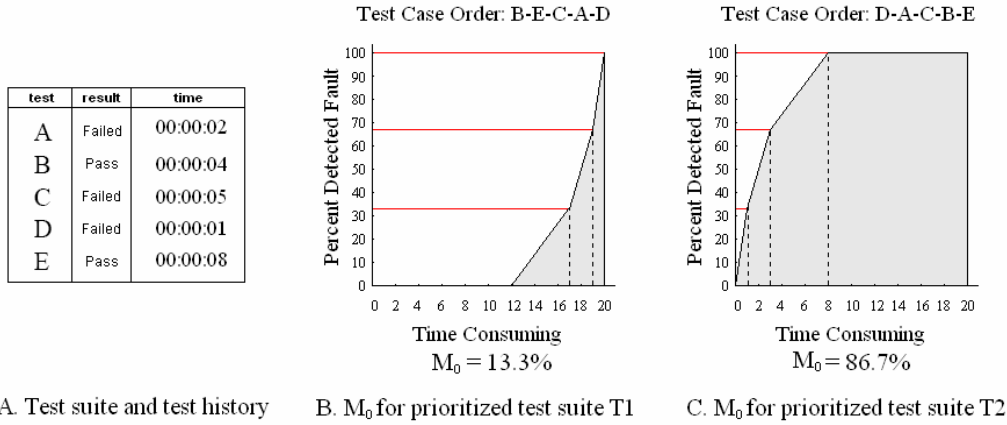


Figure 7. Example illustrating the M_0 measure

20th second. The value of M_0 here is 13.3%. And when test order is *D-A-C-B-E*, the M_0 value is 86.7%. So we say the efficiency of test suite *T2* is better than that of test suite *T1*.

Let t_i be the i^{th} test case in the ordered test suite, let $time_i$ be the execution time of t_i , and let f_i be a Boolean value that indicates if t_i is failed. M_0 for test suite *T* could be given by the equation:

$$M_0 = \frac{\frac{1}{2} \sum_{i=1}^n [(2 \sum_{j=i}^n time_j - time_i) \times f_i]}{\sum_{i=1}^n time_i \times \sum_{i=1}^n f_i} \times 100\%$$

If all the test cases have the same execution time, or the execution time is not necessary to be concerned, the equation can be rewritten as follow:

$$M_1 = \frac{\frac{1}{2} \sum_{i=1}^n [(2n - 2i + 1) \times f_i]}{n \times \sum_{i=1}^n f_i} \times 100\%$$

Our experiment will compare the rates of fault detection of different test strategies based on M_1 .

4.2. Test design

The objective of our experiments is to test the performance of Microsoft Word and PowerPoint¹ when opening malicious documents. Original test suite was generated by randomly replacing one or more bytes of normal Word (.doc) or PowerPoint (.ppt) documents. All test cases (malicious documents) were run (be opened) in previous version of target programs. If the test cases failed, including crashed or lead to a denied of service (DoS) of target programs, it would be selected for reusing.

According to test results, all failed test cases are grouped as depicted in Table 1 and 2.

Table 1. Test cases (T1) failed on Microsoft® Word® 2003(11.5529.5606)

Error Type	Test cases
Infinite loop	31
Null pointer reference	101
Illegal address reference	65
Overflow	11
Incorrect exception handling	34
Total	242

Table 2. Test cases (T2) failed on Microsoft® PowerPoint® 2000(9.0.2716)

Error Type	Test cases
Null pointer reference	289
Illegal address reference	126
Overflow	98
Illegal offset reference	177
Error in other dll	151
Other exception	95
Total	936

Here, test cases which cause a 99% CPU loading for more than 20 seconds are regarded as failed of infinite loop error. When test case was failed, a debugger would be attached to take a snapshot of fault context. Based on this snapshot, fault type was identified and test cases were grouped.

In our experiments, we test each target programs for three times. The three strategies are escalating when test case failed, deescalating when test case passed and no adjusting whether the test case failed or not, respectively. We then compared corresponding performances when different strategies were utilized and concluded.

¹ "Windows", "Office", "Word" and "PowerPoint" are registered trademarks of Microsoft Corporation.

4.3. Experiments and results

We have run all test cases on the Microsoft Office® 2003 with the newest service packet (SP2) and all hot-fixes. The general test results are show as below:

Table 3. Results of T1 on Microsoft® Word® 2003(11.8106.8107)

Error Type	Failed	Passed
Infinite loop	31	0
Null pointer reference	30	71
Illegal address reference	34	31
Overflow	0	11
Incorrect exception handling	16	18
Total	111	131

Table 4. Results of T2 on Microsoft® PowerPoint® 2003(11.8106.8107)

Error Type	Failed	Passed
Null pointer reference	101	188
Illegal address reference	19	107
Overflow	2	96
Illegal offset reference	50	127
Error in other dll	55	96
Other exception	24	71
Total	251	685

Table 3 and 4 reveal that the general failed rate of T1 on Word (45.9%) is higher than that of T2 on PowerPoint (26.8%). This is due to that there are more modified versions and more hot-fixes implemented on the latter one, therefore, most regression faults were fixed. It is also clear from tables that failed rate of each fault type values 0% to 100%. Some critical faults which may cause serious security problem have been corrected, such as overflow (100% and 98.0% respectively). However, some less important errors still remain.

In our experiments, T1's initial order was random. T2 has been run on PowerPoint XP(10.2623.2625) and

PowerPoint 2003 SP1(11.6361.6408), and then was simply prioritized based on failed times of each test cases. We then ran T1 and T2 on corresponding programs with different adjusting strategies for three times. The results are showed in Figure 8.

The sensitive constant α we used in experiments is 3. That means when one test case failed or passed, priorities of its related test cases would be escalated or deescalated with the maximum range of 3. As observed in Figure 8, M_f values for T1 are 45.2% (no adjusting), 50.1% (escalate) and 49.0% (deescalate) respectively. M_f values for T2 are 55.0% (no adjusting), 60.8% (escalate) and 60.6% (deescalate) respectively.

The M_f values of Escalate and Deescalate algorithms are 50.1% and 49.0% in T1, while 60.8% and 60.6% in T2. When no adjusting algorithms are implemented, the M_f values are 45.2% in T1 and 55.0% in T2. There is only about 1% difference between the performances of two algorithms. However, the average M_f values for dynamic adjusting strategies are about 10% higher than that of no adjusting strategies. Based on these data, we could conclude that the dynamic adjusting methods are of benefit to improving the test suite's fault detection rate in black-box testing.

In a resource-constrained environment, regression testing can seldom run all test cases. In such situations, more failed test cases are helpful for fault localization and debugging. Our algorithms can also achieve better performance when regression testing ends without executing all test cases. For example, if only 60% test cases in T2 could be run due to resource constraint, normal strategy could detect about 150 faults. However, the dynamic adjusting strategies could detect about 180 faults. Our algorithms can help to detect about 20% more than that when no adjusting algorithm is implemented.

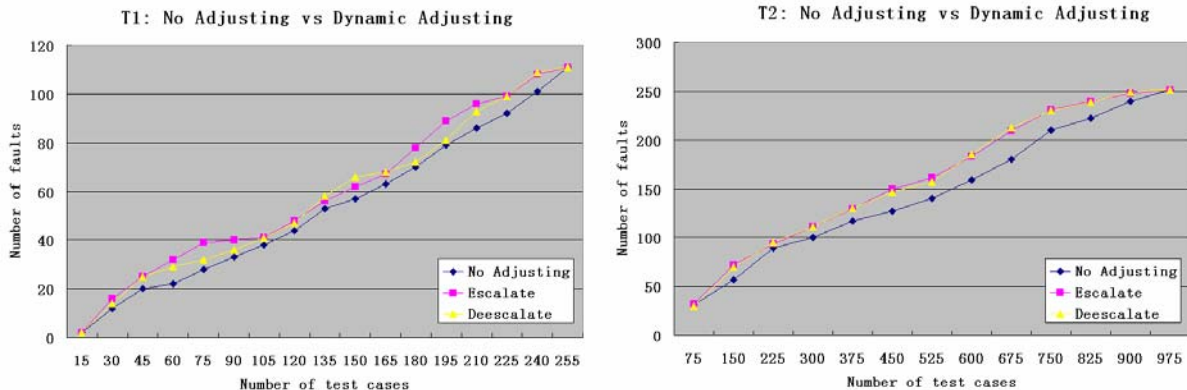


Figure 8. Test results

5. Conclusion and future work

In this paper, we describe a general process of test case prioritization for black box regression testing. To implement it, we also propose some corresponding algorithms and perform an experiment to compare the effectiveness of different test strategies. The experiment data also demonstrate that our approach can improve the rate of fault detection when performing regression test in black box environment.

As described in 3.1, two main problems of our prioritization method for black box testing are finding algorithms for constructing and using relation matrix R . Although we have presented effective algorithms, it is still possible to improve our work in the following aspects:

1. The algorithm we present is based on faults types. However, there are other kinds of information that can be utilized to calculate the relation matrix. For instance, the designing information of test cases could be used to construct R by grouping test cases that have different test objective. Future work should consider the extension of our algorithms for building matrix R incorporating different information.

2. Once the R is built, it would be a fixed matrix during the regression testing phase. A method to improve our definition is to refine R by utilizing run-time information when each test case is tested. Therefore, another direction of our future work is to research new definition of relation matrix R and its updating algorithms.

3. Our method of using R focuses on the orders of each test case, so the algorithms are nonnumeric. In future work, we are going to research numeric algorithms to adjusting selection probabilities of test cases. And we will perform experiments to compare algorithms with different adjusting granularities.

References

- [1] S. Elbaum, A. G. Malishevsky and G. Rothermel. Prioritizing test cases for regression testing. In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 102-112, August 2000.
- [2] S. Elbaum, A. Malishevsky and G. Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of the 23rd International Conference on Software Engineering*, pages 329-338, May 2001.
- [3] S. Elbaum, A. G. Malishevsky and G. Rothermel. Test case prioritization: a family of empirical studies. *IEEE Transactions on Software Engineering*, 28(2):159-182, February 2002.
- [4] S. Elbaum, G. Rothermel, S. Kanduri and A. G. Malishevsky. Selecting a cost-effective test case prioritization technique. *Software Quality Journal*, 12(3):185-210, September 2004.
- [5] J. A. Jones and M. J. Harrold. Test-suite reduction and prioritization for modified condition/decision coverage. In *Proceedings of the International Conference on Software Maintenance*, pages 92-101, October 2001.
- [6] J. M. Kim and A. Porter. A history-based test prioritization technique for regression testing in resource constrained environments. In *Proceedings of the 24th International Conference on Software Engineering*, pages 119-129, May 2002.
- [7] N. Mansour and K. El-Fakih. Natural optimization algorithms for optimal regression testing. In *Proceedings of Computer Software and Applications COMPSAC '97*, Washington, DC, USA, Pages 511-514, 1997.
- [8] G. Rothermel and M. J. Harrold. Analyzing regression test selection techniques. *IEEE Transactions on Software Engineering*, 22(8):529-551, August 1996.
- [9] G. Rothermel, M. J. Harrold. A safe, efficient regression test selection technique, *ACM Transactions on Software Engineering and Methodology*, Vol. 6, No. 2, Pages 173-210. April 1997.
- [10] D. Jeffrey and N. Gupta. Test Case Prioritization Using Relevant Slices. In *Proceedings of Computer Software and Applications COMPSAC'06*, Chicago, USA, Pages 411-420, 2006.
- [11] G. Rothermel, R. H. Untch, C. Y. Chu, M. J. Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929-948, October 2001.
- [12] H. Srikanth. Requirements-based test case prioritization. Student Research Forum in 12th ACM SIGSOFT International Symposium on the Foundations of Software Engineering, 2004.
- [13] A. Srivastava and J. Thiagarajan. Effectively prioritizing tests in development environment. In *Proceedings of the International Symposium on Software Testing and Analysis*, pages 97-106, 2002.
- [14] K. R. Walcott, M. L. Soffa, Gregory M. Kapfhammer and Robert S. Roos. Time-aware test suite prioritization. In *Proceedings of the 2006 International Symposium on Software testing and Analysis*, page 1-12, July 2006.
- [15] W. E. Wong, J. R. Horgan, S. London and H. Agrawal. A study of effective regression testing in practice. In *Proceedings of the 8th IEEE International Symposium on Software Reliability Engineering (ISSRE' 97)*, pages 264-274, November 1997.